Nye Thomas, hi24817

Ananyalakshmi Srinivasan, ud24421

1 Parallel Implementation

1.1 Functionality and Design

1.1.1 Functionality Implemented

Our initial implementation was serial; only leveraging the *distributor* goroutine to read an input PGM file as a 2D byte slice, completing the total number of turns GOL (Game of Life) in *Params* in a single for loop on this slice, and output the file as PGM. As the 2D byte slice was closed domain, we initially utilised the modulo function to calculate neighbours that 'wrapped around' the world but then decided to use if statements to count neighbours to increase efficiency. We leveraged the *IO* goroutine through channels as established in the skeleton to input and output these files as PGMs.

We then began work on our parallel implementation. This involved creation of a *worker* goroutine within our *distributor.go;* for each turn, and for the number of threads specified, the GOL image would be divided up (by the number of threads) into subworlds that would be given to these *worker* goroutines. We would initially dynamically create and destroy new *worker* goroutines for each turn to do GOL work on the respective subworlds, but then we implemented a worker pool to pre-define the no' of workers to equal the no' of threads, reusing them on each turn and synchronising them after each turn with a new copy of the whole world (built from their subworlds. This would occur at the beginning of each turn, as our core for loop would iterate within the main *distributor* goroutine). We also used a job pool to send the subworld to each goroutine. *Workers* were then gracefully closed when all turns were finished.

A *ticker* goroutine is initialised before the first turn and begins an infinite for loop; within it, a select statement that waits for either a 'done' channel signal (to indicate that all turns have finished and will break the loop) or a 'tick' on the ticker, in which a count of alive cells event is sent. Within the turns for loop body, there is a select statement that, upon receiving a 'ready' signal by the ticker, will send it the current world and turn to prevent deadlocks & data races with the main GOL computation.

The select statement also listens for keypresses sent across the *keyPresses* channel (defined in *loop.go*). The 'p' keypress will lock the user in the encompassing *for pauseLoop{}* that encompasses the select statement and will break out once 'p' is received again (using local Boolean signals). 'S' will save the current board as a PGM, and 'q' will signal a Boolean to stop executing at the end of the turn

1.1.2 Problems Overcome

Our first issue we encountered was overhead from our modulo implementation of counting neighbours. While it cleanly calculated cells in the wrap-around, the modulo function was computationally inefficient, and so we implemented if-statements instead to manually calculate the neighbours.

We also where first unsure about how separate workers could 'access' the world board and deal with edge cases (e.g. the edges of the subworlds that would interact with the other worker's subworlds). We initially considered using a 'halo exchange' solution to deal with this, but in order to prioritise time and workload we opted for a channel based solution; each worker would receive a copy of the new world through a channel(stitched together with the subworlds from the previous turn) and would operate on each cell in context with this world, being able to access neighbours operated on by other workers.

We also encountered issues with data races; mainly when working on our keypresses and ticker steps. This was mainly due to our goroutines now needing to communicate between each other further. The ticker goroutine, for example, would need to signal to the main distributor goroutine when it needs to access the current turn and world (for output). We created a data-race free solution by asking the ticker goroutine to send a signal down a *tickerReady* channel to momentarily intercept GOL execution (in its own respective select statement that would default to scheduling a GOL turn) and ask for the current turn and world (to be sent down channels connected to the ticker goroutine). A similar approach was adopted for keypresses.

1.2 Testing and Critical Analysis

1.2.1 Acquiring Results

All tests were repeated locally 5 times on a 10-core 12 thread Windows laptop. Care was taken to minimise other applications open to reduce CPU load and maintain consistency. All tests were completed on the same day for further consistency.

1.2.2 Serial vs Parallel

Figure 1 demonstrates the disparity in performance between the serial and parallel implementations over multiple trials. Notably, for 1000 turn samples, the serial implementation was 3.7x slower than parallel on average, showing an exponential increase in mean *seconds per operations* when parallel workload distribution is used.

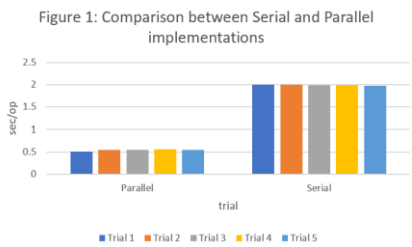

Figure 1: 5 trials conducted for both serial and parallel (16 threads), 1000 turns, 512x512 images

Figure 2 demonstrates this effect further; we can visualise the exponential throughput that is gained by partitioning work to more workers through the contrast in the yellow line (16 threads) and the previous lines (8, 4 threads respectively). It accentuates the scalability of our parallel solution when utilising additional threads.

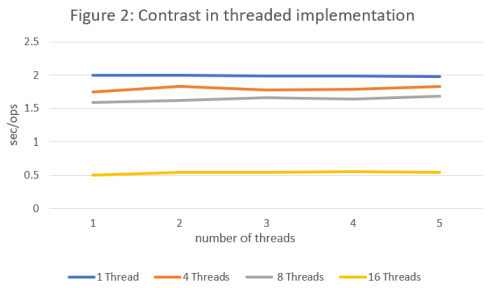

Figure 2: 5 trials conducted for all no' of threads (512x512, 1000 turns)

1.2.3 Modulus Operator Considerations

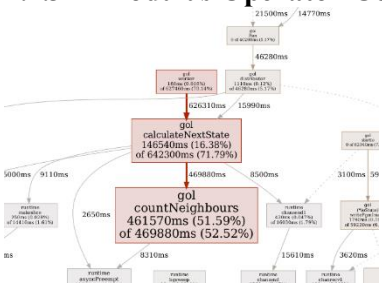

Figure 3: CPU profile of our parallel implementation utilising the modulus operator in 'countNeighbours'

During testing, we noticed significant overhead that was impacting our implementation's throughput. We generated a CPU profile (Figure 3) of our implementation to assess how CPU time was weighted across our program. Our *countNeighbours* function indicated a 51.59% hoarding of CPU time, suggesting that our implementation was causing significant overhead. We narrowed this to our use of the 'modulo' operator, a computationally costly operation that we repeated many times in the 'wrap-around' logic of our neighbour accounting. This is what prompted us to develop a solution relying more on basic if statements to avoid such overhead.

Figure 4 represents the comparison of the modulo and non-modulo implementation's performance. On average – across all threads – the non-modulo implementation was 1.8x faster than the modulo counterpart. This demonstrates the significant bottleneck implied by the modulo operation; to include it further would have heavily anchored the throughput of our parallelism attempts.

What is further important to note in Figure 4 is that both implementations follow Amdahl's Law. We can note that around roughly 10 threads both implementations begin to get *slower*. This appears to show that the cost of then inherently serial operations in our implementation begin to outweigh the cost reduction of parallelisation after a certain amount of parallelisation is achieved, regardless of if the implementation uses the modulo or not.

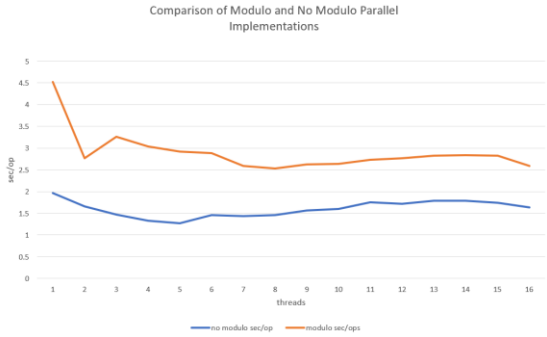

Figure 4: Comparison of modulo and non-modulo implementations from 1 – 16 threads (512 x 512, 1000 turns)

Figure 5 signals the CPU time gain across our program after implementing a non-modulo solution. We were pleased to see that the percentage CPU time spent in *countNeighbours* had dropped by 30.72%. This implied the significance of the overhead bottlenecking our previous system.

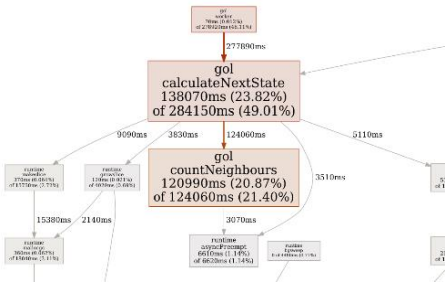

Figure 5: CPU profile of our parallel implementation without utilising the modulo operator

1.2.4 Worker Pool Considerations

We noted that our initial GOL implementation made use of an influx of goroutine construction and destruction; on each turn, we would send off new goroutines to compute the next state and allow the old goroutines to 'die'. We recognised that this would not be the most elegant or idiomatic solution and felt it would be more appropriate to designate a fixed number of worker goroutines (based on no' of threads) to be active through GOL calculation, just reusing them when necessary. This is what spurred the creation of our worker pool.

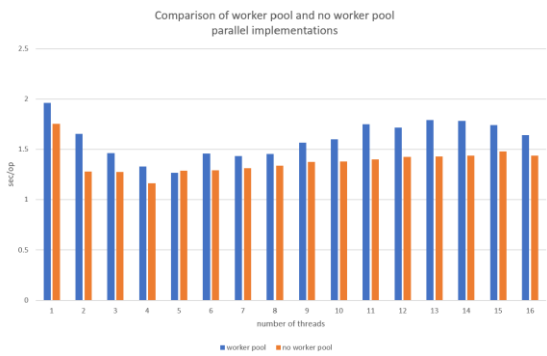

Figure 6: Comparison of worker pool and no worker pool (512x512, 1000 turns)

Figure demonstrates the comparison testing of our worker pool and lack of worker pool implementations. It suggested that, on average, our worker pool implementation was 1.16x slower than our non-worker pool implementation. This showed that there were flaws in how we implemented the worker pool; our method of thread usage optimisation was likely adding more serial wait time to our computation, so much so that it outweighed the cost of goroutine creation and destruction in our initial implementation.

Furthermore, we noted diminishing returns as the number of threads increased, especially in our worker pool implementation. This confirmed our suspicions of added serial overhead – the more threads which were utilised, the more time was spent waiting to synchronise each with each other due to our inefficient approach. It exaggerated the already existing diminishing returns of using an excessive number of threads due to Amdahl's Law.

1.2.5 Potential Improvements and Conclusion

The main improvement we would exhibit in another solution is fixing our worker pool. Our main issue was the serial handling of our worker's synchronisation; it introduced extreme serial overhead, especially when using more threads. We would approach this again by introducing an ID system for workers that would allow workers to construct their part of the next world state when they were ready, avoiding the serial for loop that would attempt to collect computation from the first worker to the last.

Another consideration would have been to pursue a halo exchange approach between goroutines. This was the nature of our first attempt, and if we had research the approach further, we believe we could have had a much more efficient communication cycle between workers.

We also would have liked to approach a shared memory solution. Given our solution was primarily a channel approach, it would have been advantageous to attempt a shared memory solution as to consider the time-gain benefits from our solution compared to channels. However, we explored this more in our distributed solution.

Overall, we were able to make significant gains in computation time through changes to costly functions (such as the modulo operator) and experimented with resource management (worker pool) to attempt time gains. We were able to witness how utilising multiple threads efficiently can exponentially decrease computation time, but with diminishing returns due to the minimum serial overhead required in our implementation.

2 Distributed Implementation

2.1 Functionality and Design

2.1.1 Functionality Implemented

The first part of our distributed implementation took the stage 1 serial code from parallel and split all Game of Life (GOL) logic to calculate next state into a *server.go* file.

We then implemented an RPC handler that which would complete all GOL turns. Then we established a request, and responses struct in a *stubs.go* file.

In order to reduce coupling, we next implemented a *broker.go* file which would receive a request from the client and then pass that to the server. Figure 7 summaries the functionality of our implementation.

We further implemented a keypresses system which would effectively pause, quit the local controller, and kill all remote components when pressing their respective keybinds, using further RPC calls and a client-side goroutine to detect keypresses as gathered from the *keypresses* channel created in the skeleton.

We implemented a ticker which similarly operated as a constant client-side goroutine, sending an RPC request to the broker to request relevant ticker data on each tick before performing the tick as required.

We then created a system that, given a global list of servers, would dial each of them. For each turn, the broker would then designate an equal slice of the world to each worker to calculate the next state, rebuilding the computed world in broker. Upon finishing all turns, this final world would be passed as response back to the client.

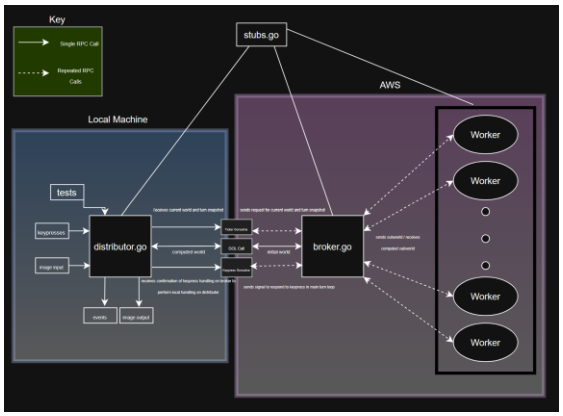

Figure 7: Diagram demonstrating data flow of our distributed solution

2.1.2 Problems Overcome

Our first issue was synchronising the turn count between the client and broker. For the ticker to report the correct number of alive cells with the corresponding turn, the client needed to receive the turn count from the broker whilst gol processes are happening seamlessly.

To allow this we created a *globals* struct which included of the *world*, *currentTurn* and a *boolean* flag which monitored whether the game was paused or not. We used mutex locks whilst accessing these to prevent multiple go routines from accessing them simultaneously.

We also opted for a RWMutexes instead of generic mutex locks to improve performance. This meant multiple go routines which only required read access could read the value of a global variable with blocking each other. They only have to wait for writers that are holding the lock.

One of our main challenges was ensuring all server nodes were shutdown gracefully when keypress 'k' was made. We found that only some of the nodes were terminated and our broker would exit with a runtime error. The cause

of this was that a *Shutdown RPC call* from the broker would often be dispatched in the middle of a GOL process. However, once a server got shutdown the response to *CallGolWorker* was empty and only an error got returned. Hence, we realised we had to return if the *CallGolWorker* RPC call returned an error as this error told us the connection to the server has been lost which is what happens when we 'kill' the server

2.2 Testing and Critical Analysis

2.2.1 Acquiring Results

All tests were repeated 5 times with the client on an 8-core MacBook Pro which was using approximately 117 threads to run Goland (in which I was running my client code) and the server running on m5.large AWS nodes.

Each plotted result is an average of these 5 outputs.

Care was taken to minimise other applications open to reduce CPU load and maintain consistency. All tests were completed on the same day for further consistency.

2.2.2 Local Servers vs Remote Servers and Broker Considerations

Upon completing a shared subworld solution for distributed, we ran benchmarking tests on both local and remote AWS nodes as to compare the networking overhead RPC calls were having on our system. Figure 8 demonstrated the stark diminishing returns of sharing work between multiple remote servers. This demonstrated a significant amount of network overhead apparent in our system, likely due to an overreliance / overuse of RPC calls and/or client connections. We hypothesised it is a similar situation to our worker pool overhead in the parallel implementation; the network overhead of adding additional nodes outweighs the parallel gain of using multiple nodes to compute separate parts of the GOL state. It also demonstrates that the subworld solution for a remote distributed GOL implementation may be starkly more inefficient than its local equivalent, as communicating such large slices of data many times remotely will cause significant network bottleneck.

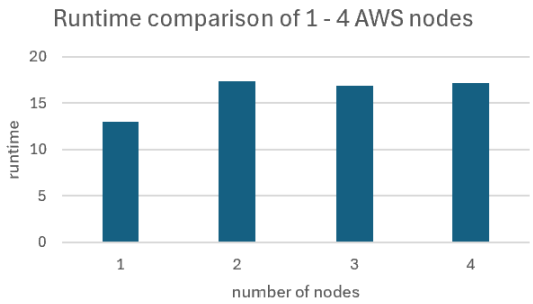

Figure 8: Graph demonstrating sec/ops as the number of nodes increase using AWS servers. 512x512, 100 turns

Figure 9 furthered our initial hypothesis, showing that when run purely on local servers, our GOL implementation shows that runtime is faster as the number of nodes increase. This is the expected outcome for an increase in worker nodes, and notably, the overall runtime is roughly 24x times faster on average than the remote benchmarking suggests. We concluded from this that network overhead was our single largest flaw in our distributed implementation at this moment to an almost anomalous extent, and initially, we were unable to effectively study our implementation as to why. However, the expected effect of the local runtime serves to prove the soundness of our implementation's GOL logic but failed to prove the efficiency of our network handling.

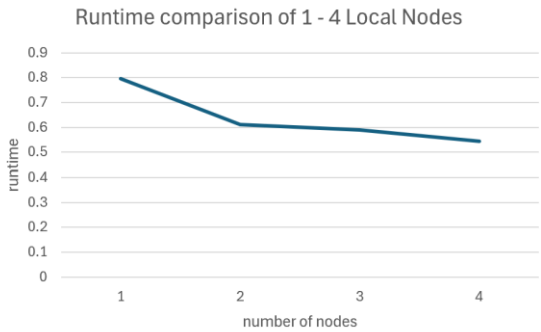

Figure 9: Graph demonstrating runtime as the number of nodes increase using local servers. 512x512, 100 turns.

However, we decided to inquire further into why our initial implementation had such anomalous results. We concluded that our *broker* goroutine, which handled the main computation loop between the remote servers, should have been hosted remotely, as then the data shared on the network would locally be 'nearer', limiting overhead. We tested this hypothesis in Figure 10, which demonstrated the expected outcome as modelled in our local server hosting. This anomaly thus became an exercise in understanding how we must temper where we are making our RPC calls, and to be aware of the volume of calls / data when a certain component is not local to the components handling these calls.

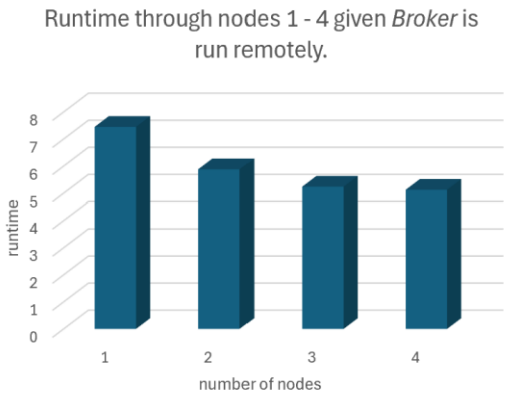

Figure 10: Graph demonstrating sec/ops as the number of nodes increase using remote servers, now running our broker on the remote system. 512x512, 100 turns.

It is further insightful to note that diminishing returns regarded by splitting work between threads on single computer are much starker than diminishing returns observed in distributed, remote implementation, as shown by Figure 11. This is however inconclusive, as due to time constraints, we were only able to test up to 4 workers on our distributed implementation, and 100 turn tests do not wholly quantify the variance of runtime seen in parallel testing. Regardless, initial testing suggests that our distributed solution could get *more* effective *quicker* than our parallel solution.

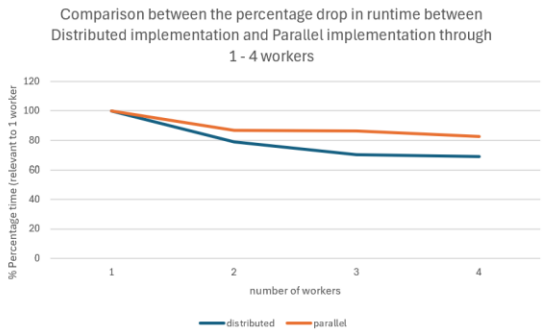

Figure 11: Comparison of percentage drop in runtime of distributed and parallel across 1 – 4 workers, 512x512, 100 turns.

2.2.3 Modulo Operator Considerations

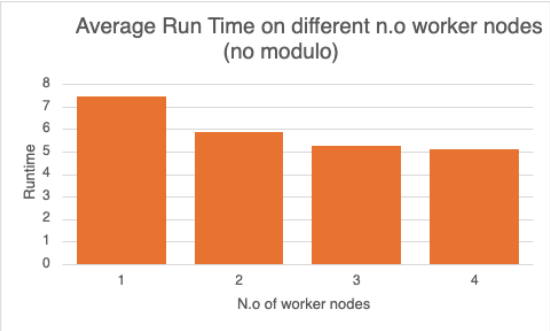

Figure 12: Comparison of runtime over multiple worker modes (no modulo) 512x512, 100 turns

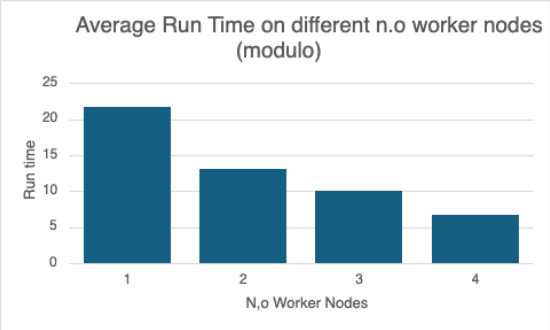

Figure 13: Comparison of runtime over multiple worker modes (modulo) 512x512, 100 turns

Just as was done in our parallel implementation, we also sought to compare the effect of the modulo operator in our distributed. As expected, Figures 12 and 13 show us the significance of the modulo operator on runtime; drastically decreasing performance. Figure 14 illustrates this further, showing the disparity in speed between the two implementations. The no modulo implementation is more quickly able to achieve a lower runtime than the modulo implementation, likely because of the added cost of the function.

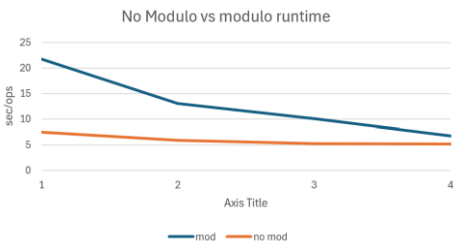

Figure 14: Line graph comparison of modulo and no modulo implementations, 512x512, 100 turns.

2.2.4 SDL Extension Consideration

We were able to develop a primitive SDL extension of our distributed implementation. The core implementation revolved around an SDL goroutine that would constantly request the turn state from the server, receiving all flipped cells not yet received up to that point and sending off the current turn state and cells flipped events. A major challenge with the solution was creating an RPC system client-side that could constantly poll to check if the current turn had progressed, given that the turn loop was being complete on the broker. The server would then provide the flipped cells via RPC from the broker, while the broker would populate a response back to the client with the current server turn, the flipped cells received from server, and the next turn to progress to client-side (turn + 1). This, however, created a system of almost constant data being shared in a through line from server to broker to client, unlike ticker and keypress, which only requested data under certain circumstances (on a poll, when a key is pressed). This created 'lag' in the client side turn polling system, and we hypothesise that the client and server turn counters are never quite synchronised, obviously resulting in incorrect alive cell counts for each turn displayed on the SDL. Nevertheless, while an incomplete system, SDL is displayed concurrently which game progression with only some added overhead. Figure 12 shows us that, on average, the SDL implementation was 2.2x slower than the non-SDL implementation, given the added network overhead. Noticeably, the SDL implementation begins to bottleneck further with the more remote worker nodes added, hinting at the exponential cost of the RPC polling throughline from client to server, with the broker now having to poll 4 separate workers near constantly.

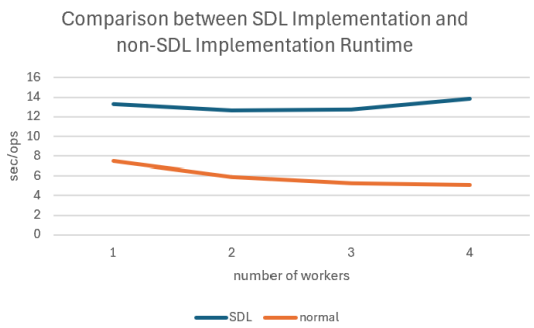

Figure 15: Comparison between SDL and non-SDL implementations from 1 – 4 workers, 512x512, 100 turns.

2.2.5 Fault Tolerance Extension

We also attempted adding Fault Tolerance to our code which would allow a new controller to take over the previous client's GOL Process. Our method was to create a .txt file to store the key state variables that will be needed by the next controller to resume the GOL process including: turn, imageHeight and imageWidth and the output image's Filename.

We then modified our *initWorld* function in our distributor to include a condition that would check for the existence of this file, and if it existed it would load the turns, imageHeight and Width from this file. It would use the output image filename to make a copy of the image in the output file and save it to the images folder so that it can be read.

However, implementing this change led to the controller deadlocking and due to time constraints, we couldn't fix this issue. Here are the possible causes: The new image file created is not a correct readable image for the controller to populate the world. As a result, the **world** is never created, and the controller cannot progress.

Alternatively, the problem could be that the turn channel never outputs a value to distributor, so the initial executing state is never set.

2.2.6 Potential Improvements and Conclusion

A major improvement to our system would have been the finishing of a working SDL implementation. While our implementation 'appears' to work, it does not pass all the tests. For a working fix in future, we propose a system that would buffer the broker turn loop to send some form of RPC response to the client-side before the next turn, allowing the client to send the relevant cells flipped events and turn complete events. This, undoubtedly, would cause even greater network lag, which is what we sought to avoid in the first place, but nevertheless would ensure there to be zero synchronisation issues between the broker and client turn count, hopefully displaying the correct alive cells count at any given time.

Another improvement would be the finishing of our fault tolerance implementation; we would trace the source of the deadlock and eliminate it.

Further improvement would have seen the creation of a halo exchange. This would have reduced the data we would have been sending over the network considerably, decreasing computation time significantly, but given time constraints it was not feasible.

Overall, our solution became more efficient than we first anticipated, given inexperience using remote tools such as AWS and ambiguity within network overhead, and we gained confidence in navigating the tools necessary to deal with these. Being able to compare the benchmarking of our distributed solution with our parallel was insightful in showing the innate issues with either implementation, and what solutions developers managing a codebase including both would have to leverage to manage them.